

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт перспективных технологий и индустриального программирования

Кафедра индустриального программирования

ОТЧЁТ

по практическому занятию № 4

по дисциплине «Создание программного обеспечения»

Тема: «Проектирование базы данных монолитного веб-приложения:
физическая реализация, оптимизация и безопасность»

Выполнил:

студент группы ЭФБО-17-24

Васин С. А. _____

Проверил:

Макиевский С. Е. _____

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
Цель работы	3
Задание	3
1 ХОД РАБОТЫ	4
1.1 Основы физического проектирования	4
1.2 Требования и анализ данных	5
1.3 Физическая схема и операции	5
1.4 Оптимизация производительности	7
1.5 Нормализация и денормализация	8
1.6 Безопасность данных	8
1.7 Тестирование и ограничения	10
2 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ	11
ЗАКЛЮЧЕНИЕ	13
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	13

ВВЕДЕНИЕ

Цель работы

Реализовать логическую модель GPU Rent в PostgreSQL 16, выбрать типы, ограничения и индексы, измерить оптимизацию запросов, обеспечить разграничение доступа и проверить целостность при конкурентной работе.

Задание

Изучить основы проектирования БД; проанализировать требования и данные; создать физическую схему; рассмотреть индексирование, кэширование и агрегирование; обосновать нормализацию и денормализацию; предусмотреть ограничение доступа и шифрование. Дополнительно подтвердить реализацию тестами.

1 ХОД РАБОТЫ

Задания №3 и №4 совпадают по тексту. Принятое разделение: ПЗ описывает концептуальную и логическую модель, П4 — физическую реализацию, измерения и тестирование. Требования FR и NFR взяты из П2; серверная и клиентская части ещё запланированы. Полная реализация находится в SQL-миграциях, а не дублируется листингами отчёта.

1.1 Основы физического проектирования

Физическая модель уточняет хранение логических сущностей: типы, ключи, ограничения, индексы, секции и права [1]. Схема `gpu_rent` содержит 19 таблиц, 9 enum-типов, 2 представления и материализованную витрину. Связь инстансов с GPU разрешена таблицей `instance_gpus`; дата-центр определяется через ноду.

Таблица 1 — Основные сущности и физические решения

Данные и таблицы	Ключи, типы и ограничения
Пользователи, <code>users</code>	<code>bigint identity</code> ; <code>email citext UNIQUE</code> ; CHECK формата <code>bcrypt</code> ; <code>balance numeric(14, 4)</code>
ДЦ, ноды, GPU	<code>bigint identity</code> ; внешние ключи; <code>UNIQUE (node_id, slot_index)</code>
Цены GPU и хранения	<code>numeric(14, 4)</code> , периоды <code>tstzrange</code> ; EXCLUDE запрещает пересечения цен одного товара
Инстансы, выделения GPU	<code>instances.id uuid</code> ; <code>instance_gpus.id bigint</code> ; снимок цены; EXCLUDE по GPU и периоду
Потребление, <code>usage_records</code>	ключ (<code>id, period_start</code>); <code>timestampztz</code> ; месячные секции
Платежи и проводки	<code>bigint identity</code> ; платёж <code>numeric(14, 2)</code> ; уникальная пара провайдера и идентификатора платежа
Секреты и аудит	<code>bytea</code> для шифртекста; <code>jsonb</code> для деталей; внешние ключи на инстанс и актора

Для денег используется точная арифметика, CHECK исключает NaN; периоды непустые и полуоткрытые [`начало`, `конец`). UTC задаёт одинаковые границы секций. Enum ограничивает статусы, триггеры проверяют разрешённые переходы. Идентификаторы большинства таблиц генерируются `GENERATED ALWAYS AS IDENTITY`, инстансы получают UUID.

1.2 Требования и анализ данных

Главные инварианты: одна GPU не обслуживает два инстанса одновременно; чужой том или SSH-ключ нельзя подключить; пополнение и биллинг идемпотентны; баланс равен сумме журнала. Горячие чтения — каталог, активные инстансы, баланс, расходы за месяц, выручка и история операций.

Детерминированный сид фиксирует момент 30.09.2026 12:00 UTC и содержит 5000 пользователей, 20 000 инстансов, 40 658 выделений GPU, 1 184 157 записей потребления, 1 208 510 проводок и 19 754 платежа. Зафиксированная загрузка занимает 37 с при лимите 2 минуты. Сид использует почасовое потребление GPU и суточное хранение; это тестовые данные, а не фактическая производственная нагрузка.

Для 100 работающих инстансов при записи раз в минуту расчёт даёт 52,56 млн записей потребления в год. По измеренному размеру строки с индексами, около 258 байт, это примерно 12,6 ГиБ; оценка обосновывает месячные секции. Журнал проводок растёт вместе с потреблением и пока не секционирован.

NFR-01 задаёт p95 API менее 300 мс. Бюджет 30 мс на запрос БД — проектное допущение, а не измеренный p95 приложения. Аналогично частоты запросов кабинета являются оценками; нагрузочный тест всего приложения ещё предстоит.

1.3 Физическая схема и операции

Двенадцать версионизируемых миграций применяются по порядку, каждая в транзакции от `gpu_rent_owner`; версия записывается в `schema_migrations`.

Таблица 2 — Группы миграций

Номера	Назначение
0001–0002	Роли, схема, UTC, расширения <code>pgcrypto</code> , <code>btree_gist</code> , <code>citext</code> ; <code>enum</code> -типы
0003–0004	Справочники и основные таблицы, ключи, CHECK и EXCLUDE
0005–0007	Защитные триггеры, секционирование, индексы
0008–0009	Функции жизненного цикла, биллинга, пополнения, шифрования; представления
0010–0011	Аудит, права и политики RLS
0012	SSH-ключ инстанса, составной внешний ключ и параметр запуска

Составные внешние ключи (`volume_id, user_id`) и (`ssh_key_id, user_id`) проверяют владельца ресурса; аналогичные ссылки потребления согласуют его пользователя с инстансом или томом. Частичная уникальность запрещает подключить один том к нескольким живым инстансам. Особенность журнала: ссылка проводки на потребление содержит (`usage_id, usage_period_start`), но не `user_id`; владельца и сумму здесь согласует функция биллинга. Для владельца платежа существует составной внешний ключ.

`fn_start_instance` проверяет пользователя, баланс и ресурсы, выбирает одну ноду и блокирует GPU через `FOR UPDATE SKIP LOCKED`. После блокировки повторно проверяет доступность и создаёт ровно запрошенное число выделений. `EXCLUDE` по (`gpu_id, allocated_during`) независимо запрещает пересекающиеся аллокации [2]. Конкурентная проверка: из 12 запусков на 4 GPU успешны ровно 4, двойных выделений нет. Пропущенная заблокированная GPU может привести к GR001; повтор должен выполнить будущее приложение.

`fn_bill_usage` блокирует пользователя, рассчитывает неоплаченный интервал, создаёт потребление и отрицательную проводку одной транзакцией, двигает `last_billed_at`. `fn_topup` зачисляет платёж один раз; частичные уникальные индексы дополнительно защищают повторное зачисление и списание. Триггер проводки обновляет баланс. Прямое изменение баланса и изменение существующих строк финансовых журналов отклоняются правами и триггерами.

`usage_records` секционирована по `period_start`: 15 месячных секций с октября 2025 по декабрь 2026 и `default`. Первичный и уникальные ключи включают ключ секционирования [3]. `fn_ensure_usage_partition` создаёт следующий месяц; автоматический вызов планировщиком ещё не настроен. Строки вне диапазона попадают в `default`, но мешают созданию соответствующей месячной секции до переноса. Внешняя ссылка журнала не позволяет просто отсоединить секцию с используемыми записями.

1.4 Оптимизация производительности

Всего объявлено 65 индексов: 60 B-tree, включая 11 частичных, 3 GiST для EXCLUDE, по одному GIN и BRIN. B-tree обслуживает равенство, диапазоны и сортировку; частичные индексы исключают завершённые ресурсы; GIN ищет в `audit_log.details`; BRIN компактно индексирует время потребления [4]. Индексы увеличивают размер и стоимость записи, поэтому выбираются под запросы.

Результаты сохранены в `db/bench/results`, сценарии — в `db/bench`. Замеры выполнены 30.09.2026: PostgreSQL 16.15, Apple M2, 8 ГБ памяти, настройки по умолчанию, один клиент, тёплый кэш. Использован EXPLAIN (ANALYZE, BUFFERS) [5]. Это исходные измерения, не новый нагрузочный прогон.

Таблица 3 — Время до и после оптимизации, мс

Сценарий	До	После	Ускорение	Решение
Активные инстансы	1,592; полный индекс 0,072	0,015	106; 4,8 раза	Частичный индекс
Расходы за месяц	33,119; отсечение 6,922	0,016	2070; 433 раза	Секции и (user_id, period_start)
Баланс	35,543; индексированный SUM 2,918	0,008	4443; 365 раз	users.balance
Выручка за месяц / год	19,227 / 145,569	0,122 / 0,981	158 / 148 раз	mv_daily_revenue
Каталог весь / один ДЦ	2,405 / 2,316	0,282 / 0,248	8,5 / 9,3 раза	Индекс открытых аллокаций
История, OFFSET 4068	27,844; индекс 0,834	0,018	1547; 46 раз	Индекс и keyset

Расходы фильтруются непосредственно по `period_start`, без `date_trunc`: читается одна секция вместо всех. Для баланса число страниц уменьшилось с 22 059 до 11. История использует курсор (`created_at`, `id`) и читает 50 строк вместо 4118 при индексированном OFFSET; `id` разрешает одинаковые временные метки. Витрина агрегирует выручку GPU по дням, ДЦ и моделям; обновление через `REFRESH CONCURRENTLY` реализовано, расписание ещё предстоит.

Все итоговые запросы укладываются в принятый бюджет 30 мс. Микросекундные результаты шумят: надёжнее порядок выигрыша и число страниц. Отдельный прежний эксперимент показал цену баланса: вставка проводки занимает около 37 мкс вместо 4,8 мкс без триггера; `fillfactor=70`

снижает её до 27 мкс, но в миграции не внесён. Эти дополнительные замеры не хранятся в `db/bench/results`.

Кэш агрегатов внутри БД реализован. Redis, подготовленные запросы будущего бэкенда, PgBouncer, мониторинг `pg_stat_statements` и реплика чтения рассматриваются как развитие, а не работающие части стенда.

1.5 Нормализация и денормализация

Логическая модель приведена к НФБК: атрибуты зависят от ключа, повторяющиеся справочники выделены отдельно. Принятое исключение из строгой первой нормальной формы — список портов шаблона. Снимок цены инстанса является историческим фактом, а не кэшем текущей цены.

Таблица 4 — Шесть осознанных денормализаций

Данные	Причина и согласованность
<code>users.balance</code>	Постоянное время чтения; триггер суммы журнала, запрет прямого UPDATE
<code>usage_records.user_id</code>	Агрегация без JOIN; составные внешние ключи владельца
<code>quantity</code> , суммы потребления и списания	Однократный расчёт и округление; одна транзакция биллинга, уникальное списание
<code>instances.gpu_model_id</code>	Отчёт без JOIN к аллокациям; неизменяемость и триггер соответствия
<code>last_billed_at</code> инстансов и томов	Граница интервала без поиска истории; меняется функциями только вперёд
<code>mv_daily_revenue</code>	Агрегация миллионов строк заранее; обновление и сверка с сырыми данными

Таким образом, ускорение оплачивается дополнительной записью, местом и контролем согласованности; проверки сравнивают баланс, проводки и витрину с первичными данными.

1.6 Безопасность данных

Реализованы роли `gpu_rent_owner`, `gpu_rent_app`, `gpu_rent_analyst` без LOGIN, отзыв прав PUBLIC, права по столбцам и RLS: 10 политик на девяти таблицах. Приложение получает контекст `app.user_id` и `app.user_role` в пределах транзакции; без пользовательского контекста клиентские строки недоступны. Аналитик читает маскированные email и ФИО через `v_users_masked`, а не исходную таблицу.

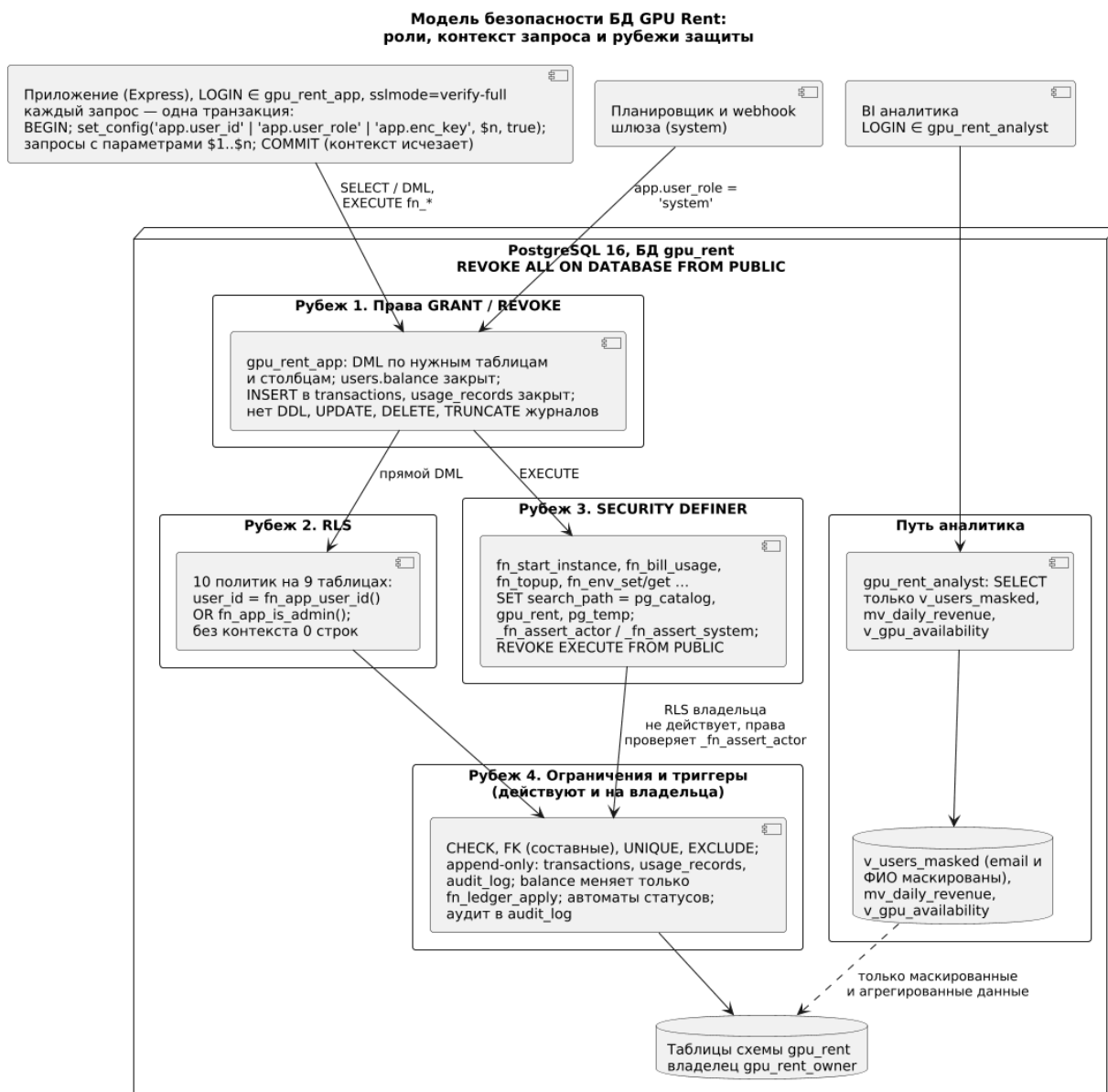


Рисунок 1 — Роли, контекст запроса и рубежи защиты БД

Пароли представлены bcrypt-хэшами, API-ключи — SHA-256-хэшами; СHECK проверяет формат, но не выполняет регистрацию пользователя. Секреты окружения шифруются pgp_sym_encrypt, ключ передаётся извне и в БД не хранится. У SECURITY DEFINER фиксирован search_path, приложение не может создавать подменные объекты [6].

Аудит фиксирует роль и статус пользователя, цены GPU и статус инстанса. Цены хранения пока не аудируются. Журнал неизменяем, но роль приложения может добавить запись с произвольным актором — это известная граница достоверности аудита.

RLS защищает от ошибок фильтрации при корректном контексте, но соединение приложения само задаёт этот контекст. При его компрометации

возможны подмена пользователя и вызов `fn_topup` или `fn_bill_usage`; нужны отдельные роли `webhook` и планировщика. Таблица `users` пока без RLS. Параметризованные запросы и белый список сортировки — требования будущего бэкенда, автоматической защиты от SQL-инъекции база не обеспечивает.

TLS, SCRAM для подключений, шифрование дисков и копий, автоматические резервные копии и PITR описаны планом, на стенде не настроены. Прежняя разовая проверка логической копии и восстановления подтвердила совпадение строк и хэшей восьми групп таблиц; это не проверка эксплуатационного расписания. При восстановлении отдельно нужны роли и параметры базы, включая UTC. Обезличивание пользователя по запросу также предстоит реализовать.

1.7 Тестирование и ограничения

SQL-тесты проверяют EXCLUDE, журнал, жизненный цикл инстанса, биллинг, пополнение, владельцев, роли/RLS, шифрование, секции, аудит, представления, сид и SSH-ключ. Тринадцать SQL-файлов содержат 414 проверок; конкурентный сценарий — ещё 14, итого 428. SQL-файлы используют фикстуры и откат, права проверяются настоящим `SET LOCAL ROLE`. Зафиксированный полный результат — PASS: 14 файлов, 428 проверок; подтверждена совместимость реализации с PostgreSQL 16.15.

Для повторения доступны скрипты применения миграций, тестов и замеров. Пересоздание базы через `reset.sh` уничтожает её содержимое, поэтому предназначено для выделенного тестового стенда. Здесь новые измерения не проводились.

Остаются протокол агента ноды и фактическое подтверждение остановки, история изменения размера тома, политика долга и накопительного округления. Хранение сейчас биллится вместе с GPU; отдельный часовой режим уменьшит число записей. Для роста нужны архивирование связанных журналов, автоматизация секций и копий, `fillfactor`, разделение привилегированных функций по ролям. Тесты подтверждают перечисленные инварианты, но не заменяют нагрузочную проверку будущего приложения.

2 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Какие основы проектирования баз данных вы усвоили? Сущности, атрибуты, связи, ключи и нормальные формы задают логическую модель. Физическая реализация добавляет типы, ограничения, индексы, секции и права.

Могли бы вы описать процесс анализа требований и определения сущностей для базы данных? Сценарии аренды, оплаты и администрирования сопоставлены с FR/NFR из П2. Из них получены пользователи, ресурсы, цены, инстансы, потребление и журнал; нагрузка определила горячие запросы.

Какие основные принципы нормализации баз данных вы понимаете? Первая форма требует атомарных значений, вторая исключает зависимость от части ключа, третья — транзитивную зависимость. В НФБК каждый детерминант является суперключом; исключения требуют обоснования.

Какие ключевые аспекты проектирования баз данных для веб-приложений вы изучили? Конкурентность, идемпотентность повторов, пагинацию, целостность и изоляцию пользователей. В GPU Rent это блокировки, EXCLUDE, уникальные проводки, keyset и RLS.

Могли бы вы рассказать о методах оптимизации производительности баз данных? Индексы уменьшают чтение, секции отсекают ненужные периоды, агрегаты и кэш избегают повторного расчёта. Эффект проверяется планом, временем и страницами до и после изменения.

Какие аспекты безопасности данных были учтены при проектировании базы данных? Минимальные права, RLS, хэши, шифрование секретов, аудит и маскирование. Защита транспорта и эксплуатационное восстановление предусмотрены планом; доверие к контексту приложения остаётся ограничением.

Можете ли вы объяснить основные концепции проектирования баз данных для монолитных веб-приложений? Единая БД позволяет объединить изменение ресурсов и финансовые записи одной ACID-транзакцией. Ограничения и атомарные функции сохраняют инварианты независимо от вызывающего модуля.

Какие методы оптимизации производительности баз данных вы рассмотрели и применили? Применены частичные и составные индексы, секционирование, материализованная витрина, баланс-кэш и keyset. Redis, пул соединений и реплика чтения рассмотрены как последующие этапы.

Какие решения по нормализации и денормализации данных были приняты в вашем проекте? Основа — НФБК с исключением для портов шаблона. Шесть денормализаций перечислены в таблице 4; согласованность обеспечивают триггеры, внешние ключи, атомарный биллинг и обновление витрины.

Можете ли вы предоставить подробное описание процесса проектирования базы данных для монолитного веб-приложения? Требования П2 преобразованы в логическую модель ПЗ, затем в 12 миграций. Сид, тесты и шесть сценариев измерений проверили схему и уточнили индексы, биллинг и ограничения защиты.

Какие методы и инструменты вы использовали для оптимизации производительности вашей базы данных? EXPLAIN (ANALYZE, BUFFERS), сценарии `psql`, детерминированный сид и сохранённые результаты. Время сопоставлено с числом страниц; стоимость денормализации проверялась отдельным экспериментом.

Какие меры безопасности данных были реализованы в вашей базе данных, и как они обеспечивают защиту информации? Права и RLS ограничивают доступ, шифрование скрывает секреты, неизменяемые журналы сохраняют историю, маскирование сокращает раскрытие персональных данных. Подмена контекста скомпрометированным приложением и эксплуатационные меры остаются открытыми задачами.

ЗАКЛЮЧЕНИЕ

Логическая модель реализована в PostgreSQL 16 через 12 миграций. Ограничения и транзакционные функции защищают выделение GPU и расчёт денег; шесть измеренных сценариев демонстрируют выигрыш от индексов, секций, агрегатов и денормализации. Проверки охватывают целостность, права и конкурентность. Реализация БД подготовлена для подключения будущего монолита; его нагрузка и эксплуатационная защита ещё не проверены.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Дейт К. Дж. Введение в системы баз данных. — 8-е изд. — М.: Вильямс, 2005.
2. PostgreSQL 16. SELECT: The Locking Clause. — <https://www.postgresql.org/docs/16/sql-select.html#SQL-FOR-UPDATE-SHARE>
3. PostgreSQL 16. Table Partitioning. — <https://www.postgresql.org/docs/16/ddl-partitioning.html>
4. PostgreSQL 16. Indexes. — <https://www.postgresql.org/docs/16/indexes.html>
5. PostgreSQL 16. Using EXPLAIN. — <https://www.postgresql.org/docs/16/using-explain.html>
6. PostgreSQL 16. CREATE FUNCTION: SECURITY DEFINER. — <https://www.postgresql.org/docs/16/sql-createfunction.html#SQL-CREATEFUNCTION-SECURITY>

Дата обращения к электронным источникам: 30.09.2026.